

Report: Analysis of the Slotted Page Layer (splayer)

1. Introduction

The splayer (Slotted Page Layer) is a fundamental component of the database storage manager, operating directly on top of the pplayer. Its primary responsibility is to manage variable-length records within the fixed-size memory blocks, known as "pages," which are provided by the underlying Paged File Layer.

This layer provides a crucial abstraction. It hides the complexities of in-page memory management, such as internal fragmentation and data-shifting, from the upper layers of the database (e.g., the Access Method Layer). It presents a clean, record-based API (e.g., Insert, Delete, Find) rather than exposing raw byte offsets. This report analyzes the core data structures and functional implementation of the splayer.

2. Core Data Structures and Concepts (sp.h)

The efficiency of the slotted page model is derived from its specific on-page data layout, which is defined in sp.h. A single page is organized into four distinct sections:

- **Page Header (SP_Page):** A fixed-size structure at the very beginning of the page.

```
typedef struct SP_Page {  
    short int num_slots; // Tracks the number of slots in use  
    short int free_space; // A pointer (offset) to the start of the free data area  
    char free_area[SP_FREE_AREA_SIZE]; // A placeholder for the slot array  
} SP_Page;
```
- **Slot Array (SP_Slot[]):** An array of SP_Slot structures that follows *immediately* after the SP_Page header. This array grows "forward" (towards the end of the page) as new slots are added.

```
typedef struct SP_Slot {  
    short int rec_len; // The length of the record  
    short int rec_off; // The offset from the beginning of the page where the record data starts  
} SP_Slot;
```
- **Free Space:** The contiguous block of memory between the end of the Slot Array and the start of the Record Data. This area shrinks from both sides as new records are added.
- **Record Data:** The actual record data is stored starting from the *end* of the page, growing "backward" (towards the beginning of the page).

The `rec_id` (Record ID) used by higher-level modules is simply the **index** of a record's

corresponding entry in the slot array.

Important Formulas

The state of the page is managed by tracking the boundaries of the free space.

- **End of Slot Array:** This offset is calculated from the start of the page.
$$\text{end_of_slots} = \text{sizeof}(\text{SP_Page}) + (\text{header} \rightarrow \text{num_slots} * \text{sizeof}(\text{SP_Slot}))$$
- **Start of Record Data:** This offset is stored directly in the header.
$$\text{start_of_data} = \text{header} \rightarrow \text{free_space}$$
- **Available Free Space:** The core formula used for insertion checks is the difference between these two pointers.
$$\text{available_space} = \text{start_of_data} - \text{end_of_slots}$$

3. Module Implementation Breakdown

3.1. sp.c - Core Page Management

- **Function: SP_InitPage(char *page)**
- **Implementation:** This function formats a raw page buffer. It casts the char* pointer to an SP_Page* pointer to overlay the header structure. It then initializes the header fields to represent an empty page:
 - $\text{sp_page} \rightarrow \text{num_slots} = 0;$
 - $\text{sp_page} \rightarrow \text{free_space} = \text{PF_PAGE_SIZE};$ (The free_space pointer starts at the very end of the page).

3.2. spininsert.c - Record Insertion

- **Function: SP_InsertRecord(char *page, char *rec, int rec_len)**
- **Implementation:** This is one of the most critical functions in the layer.
 1. **Calculate Space:** It first calculates the total space required for this new record:
$$\text{total_space_needed} = \text{rec_len} + \text{sizeof}(\text{SP_Slot})$$
 2. **Check Space:** It then calculates the available_space (using the formula from section 2) and checks if $\text{total_space_needed} > \text{available_space}$. If so, a "page full" error is returned.
 3. **Copy Data:** If space is available, it calculates the new starting offset for the record:
$$\text{new_free_space} = \text{sp_page} \rightarrow \text{free_space} - \text{rec_len};$$

It then copies the record data into this position:
$$\text{memcpy}(\text{page} + \text{new_free_space}, \text{rec}, \text{rec_len});$$
 4. **Create & Initialize Slot:** It finds the next available slot in the array ($\&\text{slots}[\text{sp_page} \rightarrow \text{num_slots}]$) and populates it with the record's metadata:
 - $\text{slot} \rightarrow \text{rec_off} = \text{new_free_space};$
 - $\text{slot} \rightarrow \text{rec_len} = \text{rec_len};$
 5. **Update Header:** Finally, it updates the page header to reflect the new state:
 - $\text{sp_page} \rightarrow \text{free_space} = \text{new_free_space};$

- `sp_page->num_slots++;`

3.3. spdelete.c - Record Deletion

- **Function: SP_DeleteRecord(char *page, int rec_id)**
- **Implementation:** The implementation uses a simple and efficient "tombstoning" method. It does not immediately reclaim space.
 1. **Find Slot:** The `rec_id` is used as a direct index to find the corresponding slot:
`SP_Slot *slot = &slots[rec_id];`
 2. **Mark as Invalid:** The slot is marked as invalid by setting its `rec_off` to a sentinel value (e.g., -1).
 - **Note on Compaction:** This simple method leaves "holes" in the page's data area. A more advanced implementation would require a costly compaction operation to shift all subsequent records and update their corresponding `rec_off` values in the slot array.

3.4. spfind.c - Finding Records

- **Function: SP_FindRecord(char *page, int rec_id, char **rec, int *rec_len)**
- **Implementation:** This function demonstrates the primary benefit of the slotted page: $O(1)$ record access time, given a `rec_id`.
 1. **Find Slot:** The `rec_id` is used as an index to get the slot: `SP_Slot *slot = &slots[rec_id];`
 2. **Check Validity:** It checks if the slot is valid (i.e., not tombstoned): if (`slot->rec_off == -1`)
 3. **Return Pointer:** If valid, it calculates the record's location using the `rec_off` from the slot and returns the pointer and length:
 - `*rec_len = slot->rec_len;`
 - `*rec = page + slot->rec_off;`

3.5. spscan.c - Scanning Records

- **Function: SP_GetNextRecord(Scan *scan, ...)**
- **Implementation:** This module provides an iterator-style interface to read all valid records on a page sequentially.
 1. **Initialize:** `SP_OpenScan` initializes a scan structure, setting the current `rec_id` index to 0.
 2. **Iterate:** `SP_GetNextRecord` iterates through the slot array, starting from its last position, while (`scan->current_rec < sp_page->num_slots`).
 3. **Check Validity:** For each slot, it checks if it is valid (i.e., `slot->rec_off != -1`).
 4. **Skip Invalid:** If a slot is invalid (deleted), it increments `scan->current_rec` and continues the loop.
 5. **Return Valid:** When a valid slot is found, it returns the record pointer and length (just like `SP_FindRecord`), increments `scan->current_rec`, and returns a "success" code.

6. **End of Scan:** If the loop finishes without finding more valid records, it returns an "end of page" code.

4. Conclusion

The splayer implementation effectively abstracts complex in-page memory management. By using a header, a forward-growing slot array, and a reverse-growing data area, it provides a robust system for handling variable-length records. This design offers the $O(1)$ access time of a fixed-array system (via `SP_FindRecord`) while maintaining the storage flexibility required for real-world data. This clean API is essential for higher-level components, such as the amlayer (Access Method Layer), which can then focus on index logic without managing page-level byte layouts.